The Clockless Detective

Student Assignment

Programming language: Python 3.11 or later

1. Problem Statement

A monitoring device displays the sequence of events `a`, `b`, `a`. The observation is consistent with two different histories: the underlying system may have operated normally, or an unobservable fault may have occurred earlier without changing the messages displayed to the observer.

The task is to determine the **maximum number of observable messages that may be generated after the first fault while the observations remain compatible with at least one fault-free execution**.

This number may be finite. A fault might instead be unreachable, or faulty and fault-free executions might produce the same observations for arbitrarily long sequences. The program must report the appropriate outcome and provide a checkable certificate: either two executions achieving the exact finite bound, or two executions containing matching repeatable cycles that demonstrate unbounded ambiguity.

The observer has no clock. Only the ordered sequence of visible messages is observed. The observer cannot detect internal steps, elapsed silent time, the current system state, or the termination of an execution.

2. Formal Model

The input is a finite-state nondeterministic transition system

$$M = (Q, \Sigma, E, s), \qquad Q = \{0, \dots, n-1\}.$$

Each transition in `edges` is a triple `(u, label, v)`.

Label	Interpretation	Observation
`a` in `alphabet`	Visible event	`a`
`tau`	Internal event	Empty observation
`fault`	Fault occurrence	Empty observation

Each transition is identified by its zero-based index in `edges`. Different transitions may have the same source and label if their destinations are distinct. Identical duplicate transitions do not occur.

Define the observation projection by

$$\pi(a) = a \quad (a \in \Sigma), \qquad \pi(\texttt{tau}) = \pi(\texttt{fault}) = \varepsilon.$$

The projection of a sequence of transitions is obtained by concatenating the projections of its constituent transitions.

The following conventions form part of the specification.

1. An execution is any valid finite path beginning at `start`. The empty execution is valid.
2. Every state is accepting. There is no separate acceptance condition; every execution prefix is admissible.
3. A missing transition means the corresponding move is impossible. Do not introduce implicit transitions or sink states.
4. Unreachable states, dead ends, self-loops, and entirely invisible cycles are permitted.
5. There are no other epsilon transitions: `tau` and `fault` are the only invisible labels.
6. A **healthy** execution contains no `fault` transition; a **faulty** execution contains at least one.
7. Faults may occur more than once. Measurement begins with the **first** fault, and returning to an old state does not erase fault history.
8. No fairness, probabilistic, progress, timing, or end-of-execution observation assumption is made.
9. Two executions with the same observation need not have the same number of transitions or end in the same state.

Let $r = e_1 \cdots e_\ell$ be a faulty execution whose first `fault` transition is e_j. Define

$$d(r) = \left|\pi(e_{j+1} \cdots e_\ell)\right|.$$

Thus $d(r)$ counts visible messages **strictly after the first fault**. Earlier messages, `tau` transitions, and `fault` transitions are not counted.

A faulty execution r is called **ambiguous** if a healthy execution h exists such that

$$\pi(r) = \pi(h).$$

Define the set of ambiguity lengths

$$\mathcal{A} = \{d(r) : r \text{ is faulty and } \exists h \text{ healthy with } \pi(r) = \pi(h)\}.$$

The program must distinguish exactly three cases:

- **`unreachable`:** No faulty execution is reachable from the initial state.
- **`bounded`:** A fault is reachable and $B = \max \mathcal{A}$ is finite. Report the **exact** value of B.
- **`unbounded`:** The set $\mathcal{A}$ is unbounded.

When a fault is reachable, $0 \in \mathcal{A}$: the faulty execution stopped immediately after its first invisible fault has the same observation as its fault-free prefix. Hence a finite bound is always nonnegative.

Interpretation of the bound. If B is finite, any faulty execution that produces at least $B + 1$ visible messages after its first fault is certainly diagnosable by that point. This does not guarantee that such a message will ever be generated: the faulty execution may stop or remain silent indefinitely.

3. Implementation Requirements

Implement the following function in `solution.py`:

```
def analyze(model: dict) -> dict:
    """Analyze the model and return the outcome and certificate."""
    ...
```

The function must not mutate its argument. On direct execution, the program must read one JSON object from standard input and write one JSON object to standard output:

```
python3 solution.py < input.json
```

Importing `solution` must not read standard input or print anything. Standard output must contain only the JSON result; debugging information should be removed or written to standard error.

Submission files:

- `solution.py`, together with any additional student-written modules. Only the Python standard library may be used.
- `tests.py`, containing at least six independent tests, at least two of which are original and distinct from the published examples.
- `report.md`, at most 1,200 words, explaining the model, correctness and termination arguments, time and space complexity, and a counterexample refuting a plausible but incorrect simpler approach.

All three cases, exact finite bounds, and certificate construction are mandatory. A graphical interface is optional. There is no requirement to output a shortest or lexicographically smallest certificate.

4. Input and Output Format

Each invocation receives exactly one JSON object with the following keys.

Key	Definition
`n`	Integer number of states
`start`	Integer identifier of the initial state
`alphabet`	Array of observable event symbols
`edges`	Array of triples `[source, label, destination]`

Inputs are guaranteed valid. The alphabet consists of distinct, single-character strings in `a` through `z`; transition labels are either alphabet symbols, `tau`, or `fault`. Every state index is in `0..n-1`. The order of `edges` determines transition identifiers but has no other semantic significance.

4.1 Case 1: Unreachable Fault

```
{"status":"unreachable"}
```

4.2 Case 2: Finite Ambiguity Bound

```
{
  "status": "bounded",
  "bound": 2,
  "witness": {
    "faulty": [0, 2, 3, 4, 5],
    "healthy": [0, 7, 8]
  }
}
```

The arrays contain **transition identifiers**, not state identifiers. Both paths must begin at `start`. The faulty path must contain a `fault` transition, and the healthy path must contain none. Their projected observations must coincide, and the number of visible messages after the first fault in the faulty path must equal `bound`.

4.3 Case 3: Unbounded Ambiguity

```
{
  "status": "unbounded",
  "prefix": {"faulty": [1], "healthy": []},
  "cycle": {"faulty": [2, 3, 4], "healthy": [0]}
}
```

Here the two `prefix` paths start at `start`, produce identical observations, and end at states from which the corresponding `cycle` paths can be traversed. The faulty prefix must contain a fault; the healthy prefix and healthy cycle must be fault-free. Each cycle must return to the state from which it started, and the two cycles must have **the same, nonempty observation**. Consequently, repeating both cycles any number of times yields indistinguishable healthy and faulty executions with arbitrarily many post-fault observations.

Across all certificate arrays, the total number of transition identifiers must not exceed `8*n*n`. A certificate meeting this limit always exists. Identifiers must be integers, and empty arrays are allowed wherever the stated validity conditions permit them. JSON key order and whitespace are immaterial, but only the specified keys for the chosen status may appear. The grader validates certificates semantically; a valid certificate need not be identical to a published example.

5. Worked Examples

5.1 Example 1. A Fault Exposed by the Next Observable Event

Input

```
{
  "n": 3,
  "start": 0,
  "alphabet": ["a", "b"],
  "edges": [
    [0, "a", 0],
    [0, "fault", 1],
    [1, "tau", 1],
    [1, "b", 2],
    [2, "b", 2]
  ]
}
```

One valid output

```
{"status":"bounded","bound":0,"witness":{"faulty":[1],"healthy":[]}}
```

Healthy executions can emit only `a` symbols. The fault itself is invisible, but the next visible `b` has no healthy explanation. The `tau` self-loop may produce arbitrary silence; it does not increase the number of ambiguous visible messages.

5.2 Example 2. One Healthy Explanation Suffices

Input

```
{
  "n": 7,
  "start": 0,
  "alphabet": ["a", "b", "c"],
  "edges": [
    [0, "tau", 1],
    [1, "a", 1],
    [1, "fault", 2],
    [2, "tau", 3],
    [3, "a", 4],
    [4, "b", 5],
    [5, "c", 5],
    [1, "a", 6],
    [6, "b", 1]
  ]
}
```

One valid output

```
{"status":"bounded","bound":2,
 "witness":{"faulty":[0,2,3,4,5],"healthy":[0,7,8]}}
```

Both witnesses project to `ab`. A different healthy choice at state 1 would fail to match `ab`, but a **single** successful healthy explanation is sufficient to preserve ambiguity. The next `c` on the faulty branch cannot be matched by any healthy run. Observable messages generated before the fault do not contribute to the bound.

5.3 Example 3. Cycles on Both Sides, but Finite Ambiguity

Input

```
{
  "n": 6,
  "start": 0,
  "alphabet": ["a", "b"],
  "edges": [
    [0, "tau", 1],
    [0, "fault", 4],
    [1, "a", 1],
    [1, "b", 2],
    [2, "a", 2],
    [2, "b", 3],
    [3, "a", 3],
    [4, "a", 5],
    [5, "b", 4]
  ]
}
```

One valid output

```
{"status":"bounded","bound":5,
 "witness":{"faulty":[1,7,8,7,8,7],
            "healthy":[0,2,3,4,5,6]}}
```

Both executions project to `ababa`. The healthy branch can contain arbitrarily many `a` symbols but at most two `b` symbols. The faulty branch produces a prefix of `ababab...` Thus five post-fault observations can remain ambiguous; the third `b` at position six reveals the fault.

5.4 Example 4. A Repeatable Certificate of Unbounded Ambiguity

Input

```
{
  "n": 4,
  "start": 0,
  "alphabet": ["a"],
  "edges": [
    [0, "a", 0],
    [0, "fault", 1],
    [1, "tau", 2],
    [2, "a", 3],
    [3, "tau", 1]
  ]
}
```

One valid output

```
{"status":"unbounded",
 "prefix":{"faulty":[1],"healthy":[]},
 "cycle":{"faulty":[2,3,4],"healthy":[0]}}
```

One traversal of the faulty cycle comprises three transitions but produces only one visible `a`. The healthy cycle also produces one `a`. Each cycle returns to its original state, allowing the observation to be extended arbitrarily many times without distinguishing the executions.

6. Algorithmic Challenge and Hints

Discovering a fault-containing path is not sufficient. The program must reason about all possible healthy explanations of an observation and make correct claims about potentially unbounded behavior. Simulating a fixed number of steps does not constitute a proof.

Hints are intended to be considered in the following order:

1. To demonstrate that the system could still be healthy, how many healthy witness executions are needed?
2. Two executions with identical observations may contain different numbers of internal steps. Is synchronization by raw transition count legitimate?
3. Which information from two long execution histories is sufficient to reason about their continuations?
4. Does every repeated state imply growing ambiguity? Distinguish cycles containing visible messages from entirely invisible cycles.

7. Constraints and Prerequisites

Parameter	Bound
Number of states, `n`	`1..80`
Observable alphabet size	`1..6`
Outgoing transitions per state per observable symbol	At most `2`
Outgoing `tau` transitions per state	At most `2`
Outgoing `fault` transitions per state	At most `1`
Total transitions	At most `n*(2*len(alphabet)+3)`, possibly zero
Running time per instance	`8 s`
Memory	`256 MiB`
Total certificate transition identifiers	At most `8*n*n`

Approaches that enumerate strings up to an arbitrary cutoff or incur exponential complexity in the number of states are unlikely to satisfy the larger tests.

In addition to regular languages and DFA/NFA theory, familiarity with BFS, DFS, strongly connected components (SCCs), and dynamic programming on directed acyclic graphs (DAGs) is useful. If SCCs and DAG algorithms have not been covered, budget approximately one hour for introductory study. An attributed general-purpose SCC implementation from course notes may be reused, but the problem reduction, reasoning, and certificate generation must be original work.

8. Required Student Tests

The six submitted tests must collectively exercise all three statuses. Consider unreachable faults, a fault on the first transition, a dead end after a fault, an entirely invisible cycle, several same-label NFA transitions, and witnesses whose two sides have different raw transition counts. For the two original test cases, explain in one sentence which plausible implementation error each test is intended to reveal.

9. Assessment

Criterion	Points
Correct status and exact finite bound in core cases	30
Valid bounded and unbounded certificates	20
Correctness on boundary and adversarial cases	15
Efficiency on larger instances, conditional on correctness	15
Code clarity, modularity, and meaningful tests	8
Technical report, correctness justification, and complexity analysis	12
Total	**100**

Eighty points are assigned by automated evaluation and twenty by code and report review. Core cases, edge cases, and performance are assessed separately; efficiency credit is awarded only for correct results.

Optional bonus (up to 5 points, beyond the 100-point scale):

- **2 points:** Prove a polynomial upper bound expressed solely in terms of `n` for every finite `B`. An unproved linear bound is not acceptable.
- **3 points:** Implement the optional visualization described below and document which parts of the certificate it replays.

10. Optional Visualization

A **Two-Realities Viewer** may replay the certificate in two panes, one healthy and one faulty. Use a simple library such as `tkinter`. Internal events, including `tau` and `fault`, should be visible only in the internal-state pane; a shared display should show the observer's visible messages. For an unbounded certificate, allow the viewer to choose a repetition count and demonstrate that the two resulting observations remain identical. Display the number of visible messages generated since the first fault.

The viewer must read the certificate emitted by the solver. It must not artificially synchronize the two sides' invisible actions. The visualization is supplementary and cannot replace the required analysis or proof.